

Лекция 01



Безопасная разработка: от требований до выпуска

**Как обосновать, что конкретную версию
программного продукта можно выпускать?**

Факультет компьютерных наук

Бакалаврская образовательная программа 09.03.04 «Программная инженерия» · 3 курс

Дисциплина «Разработка безопасного программного обеспечения» · 2026/2027 учебный год



Секция 1

Устройство курса

Что изучаем, как работаем и где появляется итоговый результат



О чем этот курс

3 / 47

Безопасная разработка программного обеспечения — разработка, в которой риски безопасности учитываются на протяжении жизненного цикла: от требований и проектирования до реализации, проверок и сопровождения.

**Обоснованность решений подтверждается
наблюдаемыми результатами работы**

Уже сегодня мы разберем как

различать инженерный артефакт, решение и свидетельство

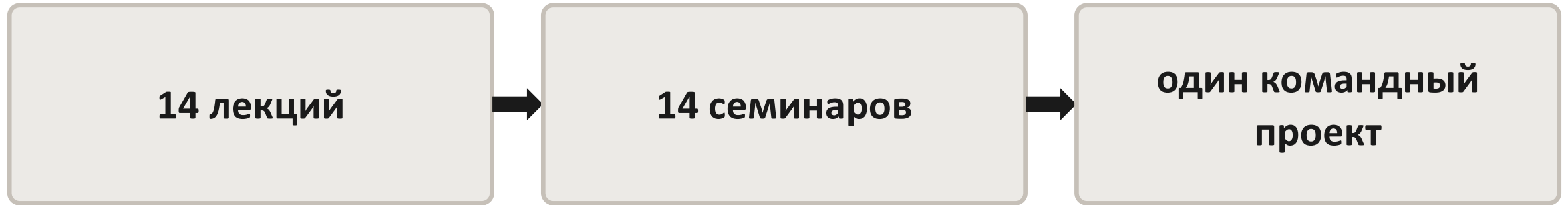
формулировать ограниченное проверяемое утверждение о продукте

объяснять путь от контекста продукта до обоснованного решения о выпуске

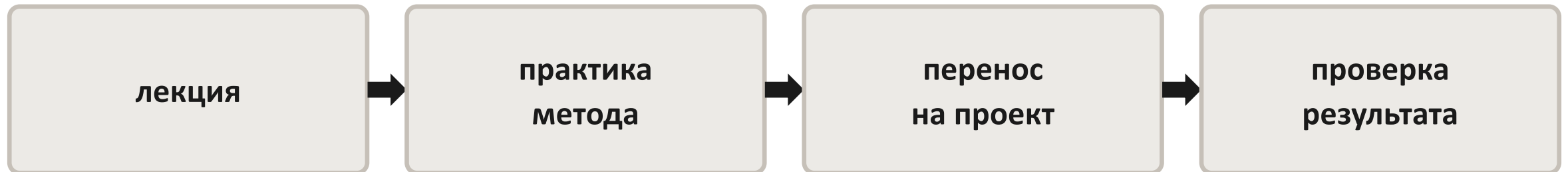


Как устроен курс

4 / 47



Повторяющийся учебный ритм





Пять разделов — один инженерный маршрут

5 / 47

1 Требования безопасности и проектирование защитных механизмов

2 Анализ и исправление дефектов

3 Автоматизация и статический анализ

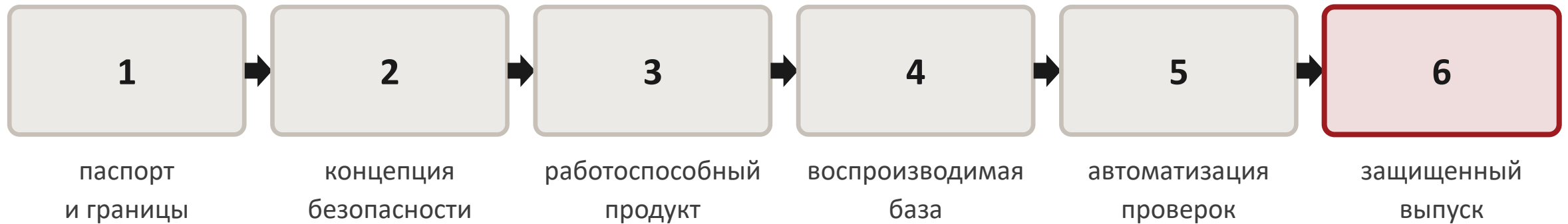
4 Зависимости, контейнеры и динамический анализ

5 Решение о выпуске



Проект проходит шесть этапов

6 / 47



Все этапы описывают один продукт и одну развивающуюся цепочку решений



Где находятся элементы контроля

7 / 47



ДЗ-1 и ДЗ-2 растут вместе с командным продуктом; КР и экзамен — индивидуальные



Как считается итоговая оценка

8 / 47

$$\text{Итоговая оценка} = 0,20 \times \text{ДЗ-1} + 0,20 \times \text{КР} + 0,30 \times \text{ДЗ-2} + 0,30 \times \text{Э}$$

**Экзамен полностью блокирующий: оценка ниже 4 из 10
не позволяет получить положительную итоговую оценку**



Где что находится

9 / 47

Где что лежит

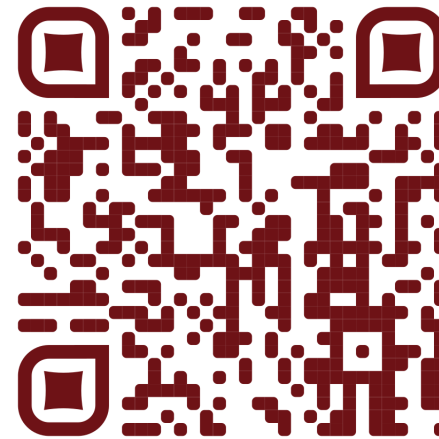
репозиторий курса — актуальные материалы

SmartLMS — дублирование и каналы сдачи

телеграм-канал — объявления и напоминания

репозиторий команды — после семинара 01

Репозиторий курса



github.com/hse-rbpo-bachelor-2026/course

Telegram-канал



t.me/rbpo_hse

Генеративный ИИ — по правилам из файла ai-usage.md (в репозитории курса).
На контрольной работе и экзамене ИИ запрещен



Секция 2

Безопасность как свойство продукта

Почему корректная программа еще не означает безопасный продукт



Что мы называем программным продуктом

11 / 47

Программный продукт

— программа вместе с конфигурацией, зависимостями, данными, средой исполнения и правилами эксплуатации, необходимыми для ее использования.

Рабочее определение курса





Свойство безопасности

12 / 47

Свойство безопасности — характеристика защищенного поведения или состояния системы; требование конкретизирует значимое свойство для конкретного продукта.

Конфиденциальность

ограничивает раскрытие информации неуполномоченным сторонам

Целостность

защищает от несанкционированного изменения или уничтожения

Доступность

обеспечивает своевременный доступ уполномоченных пользователей к данным и функциям

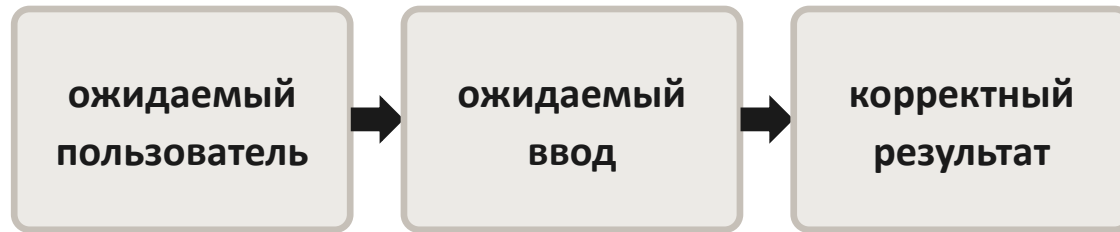
Дальше в курсе эти свойства связываются с требованиями, угрозами, решениями и проверками



Корректно — еще не значит безопасно

13 / 47

ОЖИДАЕМЫЕ УСЛОВИЯ



НЕБЛАГОПРИЯТНЫЕ УСЛОВИЯ

ошибочный или враждебный ввод

попытка выйти за полномочия

уязвимая зависимость

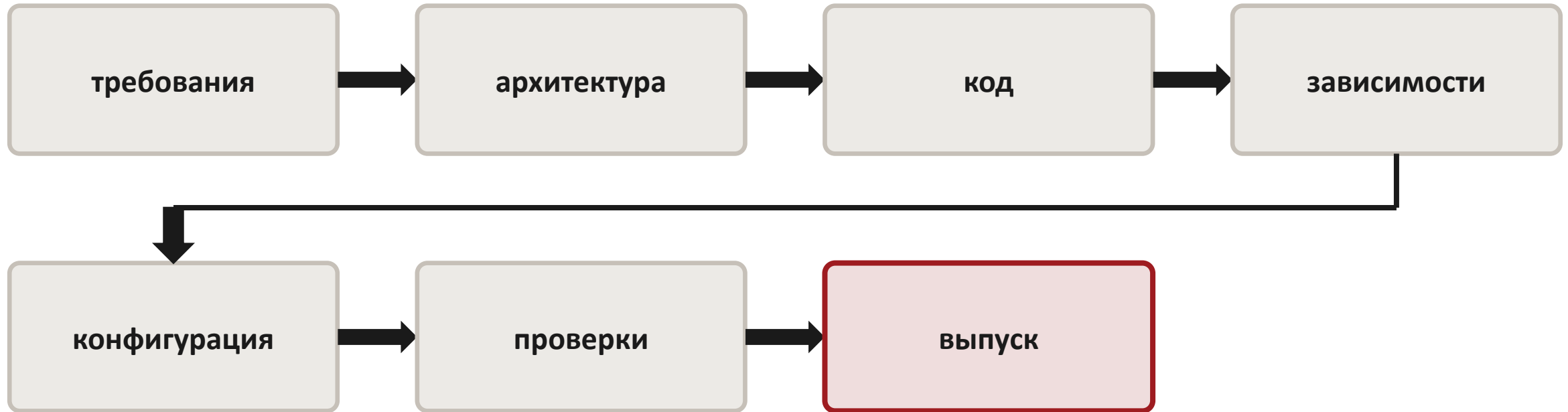
небезопасная конфигурация

**Безопасный продукт должен сохранять приемлемое поведение
и за пределами штатного сценария**



Где формируется безопасность

14 / 47



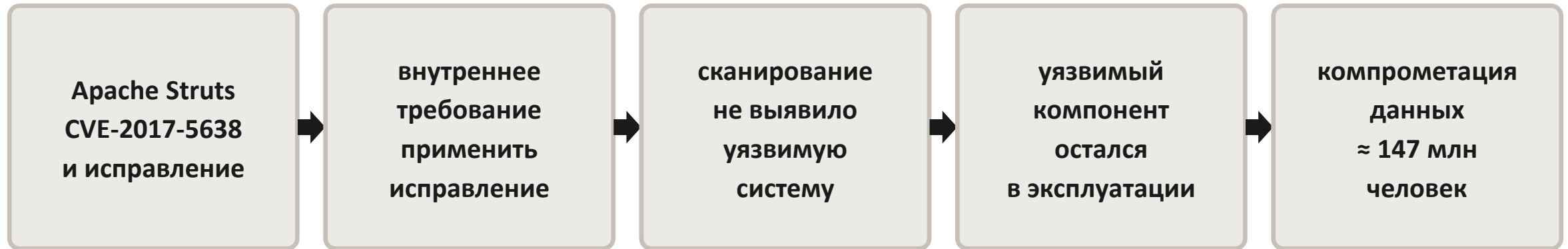
Безопасность формируется по всей цепочке разработки



Код не менялся. Продукт стал небезопасным

15 / 47

Equifax, 2017



Состояние зависимости — часть состояния продукта

Источники: U.S. House Oversight Committee, Equifax report (2018); FTC, Equifax complaint (2019); CVE-2017-5638



Почему поздняя проверка не спасает

16 / 47

нет определенной границы



непонятно, что проверять

нет проверяемого утверждения



непонятно, что считать успехом

неполное защитное решение



поздняя проверка видит лишь часть последствий

**Чем раньше сформулировано проверяемое свойство,
тем легче связать с ним решение и проверку**



За что отвечает команда разработки

17 / 47



Итоговое решение о принятии организационного риска
может принадлежать другому субъекту



Секция 3

Артефакт, решение и свидетельство

Что именно может служить основанием инженерного вывода



Артефакт, решение, свидетельство

19 / 47

Инженерный артефакт

сохраненный результат инженерной работы: код, тест, модель, схема, документ, конфигурация, машинный отчет, собранная версия, коммит или запрос на слияние

отчет

Инженерное решение

обоснованный и зафиксированный выбор действия или механизма, связывающий основания выбора с изменением продукта и проверкой

исправить

Свидетельство

артефакт или связка артефактов, значение которых обоснованно интерпретировано применительно к утверждению и связано с определенной версией; нужен наблюдаемый результат проверки

отчет + смысл + версия



Артефакт сам по себе ничего не доказывает

20 / 47

**0 проблем
обнаружено**

**Что именно подтверждает
этот отчет теперь?**

отчет относится к предыдущей версии

основной каталог исключен из анализа

конфигурация запуска неизвестна



Факт → интерпретация → решение

21 / 47





Пять вопросов к сильному свидетельству

22 / 47

1 Что утверждается?

2 Какой объект и какая версия?

3 Как получен результат?

4 Как он интерпретирован?

5 Какое решение принято?

Для повторяемости также важны область, конфигурация и сохраненный результат.



Где разрывается доказательная цепочка

23 / 47

результат	✗	без версии
тест	✗	не охватывает опасный сценарий
исправление	✗	без повторной проверки
ссылка ведет	✗	к изменившемуся результату

**Сможет ли другой участник восстановить утверждение,
результат и логику решения?**



Результат подтверждает только установленную область

24 / 47

проверен один маршрут $\neq$ проверены все маршруты

проверен исходный код $\neq$ проверена внешняя конфигурация

проверен образ $\neq$ доказано, что именно он развернут

СЛАБЫЙ ВЫВОД

«Продукт не содержит уязвимостей»

РАБОЧИЙ ВЫВОД

«Нарушений не найдено в установленной области и при выбранном способе проверки»

Equifax: автоматизированная проверка не выявила уязвимый компонент Apache Struts; по данным FTC, сканер был настроен так, что не проверял все места, где могло использоваться уязвимое ПО.



Число отчетов не доказывает полноту

25 / 47

ТРИ ОТЧЕТА

средство А: тот же сигнал

средство В: тот же сигнал

средство С: тот же сигнал

одна область подтверждена,
остальные не проверялись

ОДНА ЦЕПОЧКА

утверждение об одном ресурсе

решение и изменение продукта

проверка опасного сценария

утверждение подтверждено
в установленной области

Полноту определяет покрытие утверждений, а не количество артефактов



Хорошее утверждение можно проверить

26 / 47

«Доступ к обращениям безопасен»



«Обычный сотрудник может читать содержание и вложения
только собственных обращений»

субъект



действие



ресурс / ограничение



Секция 4

Центральный инженерный цикл

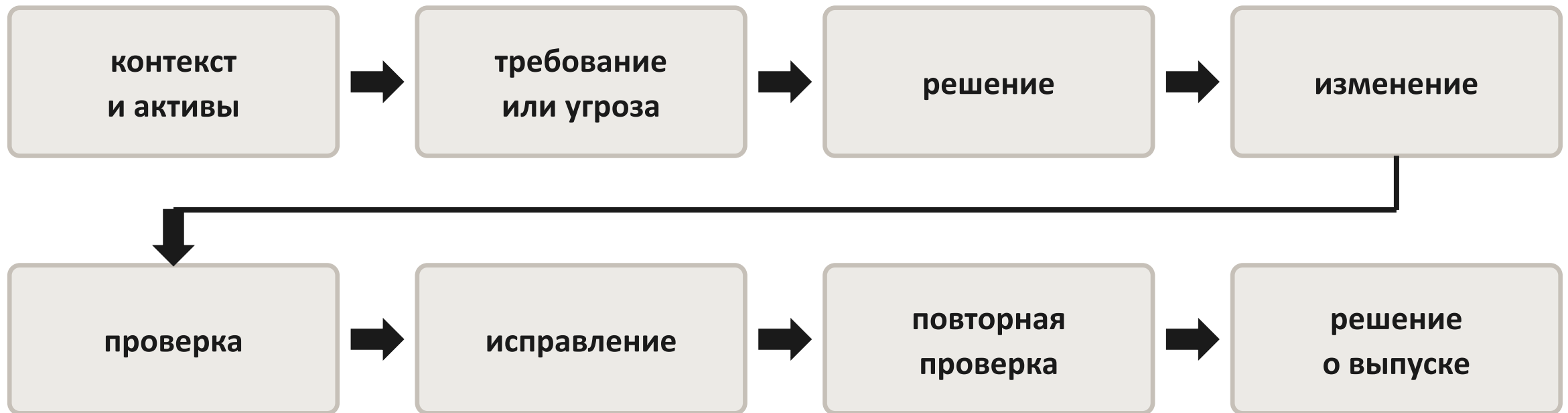
Один цикл связывает требования, решения, изменения, проверки и выпуск



Один инженерный цикл связывает весь курс

28 / 47

Центральный инженерный цикл — последовательность от контекста и значимых активов через требование или угрозу к решению, изменению продукта, проверке, исправлению, повторной проверке и решению о выпуске.



Рабочее определение курса



Цикл может начаться с разных событий

29 / 47

новое требование



релевантная угроза



автоматизированный сигнал



найденный дефект



обновление зависимости



**установить применимость
→ принять решение**



Контекст и активы задают смысл

30 / 47

Сервис обращений

пользователи

сотрудник · специалист поддержки · руководитель

значимые объекты

обращение · вложение · история обработки

неприемлемое событие

сотрудник получает чужое обращение

значимый
объект



желаемое
свойство



возможное
нарушение

Угроза — потенциальный сценарий события или действия, способного нарушить значимое свойство продукта (формальная модель угроз вводится на лекции 03)



Решение должно изменять продукт

31 / 47

ПРОВЕРЯЕМОЕ ПОЛОЖЕНИЕ

**«Сотрудник читает только
собственные обращения»**



РЕШЕНИЕ

**«Сервер проверяет право доступа
при каждом обращении к ресурсу»**

решение + предположения + ограничения

КОНТРПРИМЕР

«Скрыть кнопку
чужого обращения
в интерфейсе»

— недостаточно



Проверка создает наблюдаемый результат

32 / 47

сотрудник читает собственное обращение

РАЗРЕШЕНО

сотрудник запрашивает чужое обращение

ЗАПРЕЩЕНО

специалист открывает назначенную запись

РАЗРЕШЕНО

сотрудник напрямую запрашивает чужое вложение

ЗАПРЕЩЕНО

неуспешная
проверка



локализация



исправление



повторная
проверка



Решение о выпуске — это вывод

33 / 47



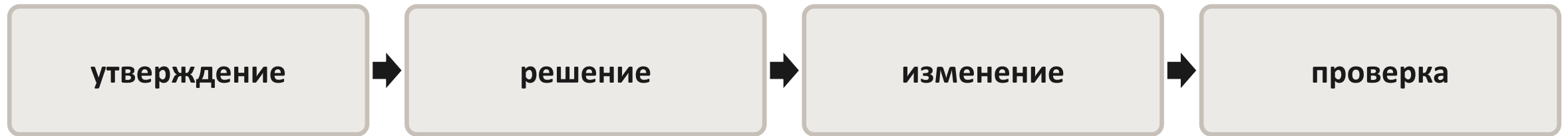


Трассируемость работает в обе стороны

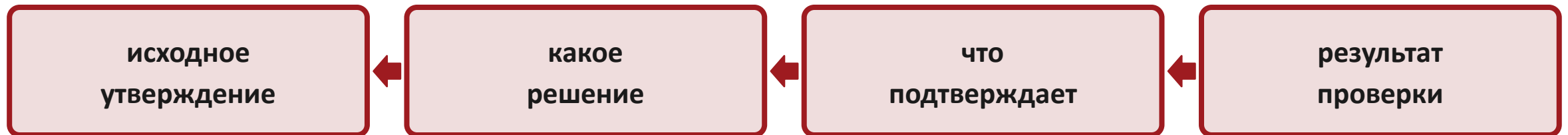
34 / 47

Трассируемость — свойство проектных материалов, позволяющее проследить, почему существует требование или решение, с какой угрозой оно связано и каким результатом подтверждается.

ВПЕРЕД



НАЗАД



Возврат к предыдущему звену не является неудачей процесса: новое знание используется до выпуска

Рабочее определение курса



Секция 5

Граница продукта

Что команда действительно может изменять и проверять



Граница продукта

36 / 47

Граница продукта — явно определенный объем продукта: компоненты, данные, интерфейсы, конфигурации и взаимодействия, за которые отвечает команда, а также существенные элементы внешнего окружения.



Рабочее определение курса



Что именно мы защищаем: актив

37 / 47

Актив — данные, функция, ресурс или состояние продукта, нарушение конфиденциальности, целостности или доступности которых способно причинить значимый ущерб.

Примеры для сервиса обращений

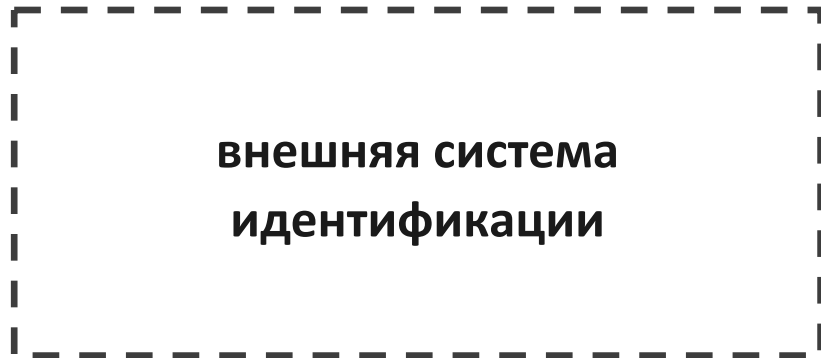




Внешний компонент влияет на продукт

38 / 47

СНАРУЖИ ГРАНИЦЫ



внешняя система
идентификации

ВНУТРИ ОТВЕТСТВЕННОСТИ ПРОДУКТА

обработать полученный идентификатор

связать его с внутренней ролью

проверить право на конкретный ресурс

корректно обработать отказ внешней стороны

не дать внешнему участнику лишних полномочий

**Команда не исправляет внешний сервис,
но отвечает за собственную сторону взаимодействия**



Как выглядит полезная граница

39 / 47

Слишком широко

«все, что относится
к приложению»

Рабочий уровень

компоненты, интерфейсы,
данные и зоны
ответственности

Слишком подробно

перечень каждой
библиотеки и мелкой
реализации



Граница меняется вместе с продуктом

40 / 47





Секция 6

Профессиональная рамка

Где эта логика находится в профессиональной практике



Два профессиональных ориентира

42 / 47

NIST SP 800-218

*Secure Software Development Framework (SSDF)
Version 1.1*

финальная редакция, 2022

**Карта высокоуровневых практик безопасной
разработки и общий профессиональный язык**

ГОСТ Р 56939-2024

*«Защита информации. Разработка безопасного
программного обеспечения. Общие требования»*

действует с 20.12.2024

**Российский нормативный контекст требований к
работам по созданию безопасного ПО и
устранению выявленных недостатков**

**Курс использует эти документы как внешние профессиональные
координаты, но не строится как их последовательный пересказ**



SSDF: четыре группы практик

43 / 47

PO *Prepare the Organization*

подготовить людей, процессы и технологии организации к безопасной разработке

PS *Protect the Software*

защищать код, зависимости и собираемые артефакты от подмены и несанкционированного доступа

PW *Produce Well-Secured Software*

создавать защищенное ПО с минимумом уязвимостей в выпусках

RV *Respond to Vulnerabilities*

выявлять остаточные уязвимости, реагировать на них и предотвращать повторение

Коды и формулировки заучивать не требуется



Как курс ложится на карту SSDF

44 / 47

PO	роли, правила работы и техническая основа
PS	код, конфигурация, зависимости и сохранность артефактов
PW	требования, решения, реализация и воспроизводимые проверки
RV	сигналы, исправления, повторная проверка и улучшение процесса

ГОСТ Р 56939-2024 — российская нормативная координата разработки безопасного ПО.

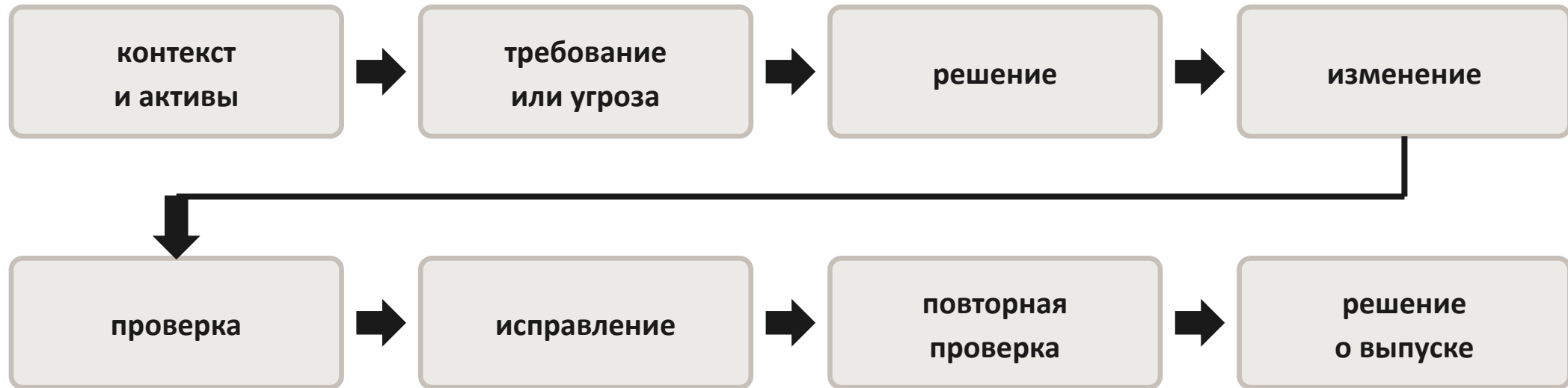
Этот стандарт подробно рассматривается на последней лекции курса

**Рамки нужны для проверки профессиональной полноты;
курс остается инженерным маршрутом вокруг конкретного продукта**



Вся лекция — в одной схеме

45 / 47



безопасность — свойство
программного продукта

граница задает область
ответственности команды

свидетельство связано с утверждением
и конкретной версией

решение о выпуске следует
из накопленных свидетельств



На семинаре 01: выбрать и ограничить небольшой командный продукт

На занятии

сформировать команду

определить назначение и границу продукта

описать пользователей

наметить не менее двух законченных сценариев

определить значимые объекты

отделить минимальное ядро от расширений

начать PROJECT.md

До семинара 01 специальная
подготовка не требуется

Пока не требуются

готовый код

технический каркас

формальные SR и модель угроз

проектные решения и анализаторы

